

RETAILMART V3 • SQL

Scalar Functions (String & Date)

WhatsApp Announcement

SCALAR FUNCTIONS (STRING & DATE)

Ever wondered how apps format 'rAhUL sHaRMa' into 'Rahul Sharma'? Or show '15-May-2025' instead of '2025-05-15T14:30:00+05:30'? Today you learn the functions behind it all.

Today's Focus:

- String — UPPER, LOWER, INITCAP, LENGTH, SUBSTRING, TRIM, CONCAT, REPLACE, POSITION
- Date — NOW(), EXTRACT, DATE_TRUNC, TO_CHAR, interval arithmetic
- Real-world data cleaning on RetailMart V3

80% of real-world SQL is cleaning and formatting data. These are the tools.

— Siraj Sir

#Day6 #PostgreSQL #StringFunctions #DateFunctions

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Quick Recap — earlier sessions	5 mins	SELECT, WHERE, ORDER BY, LIMIT — Filtering & Sorting essentials, DQL = reading data without changing it
Part 1: String Functions	55 mins	UPPER, LOWER, INITCAP — case conversion, LENGTH — counting characters, CONCAT & — joining strings, SUBSTRING, POSITION, SPLIT_PART — extracting parts, TRIM, REPLACE — cleaning text, 4 practice problems (Easy → Crazy)
Part 2: Date/Time Functions	55 mins	NOW(), CURRENT_DATE — getting current date/time, EXTRACT — pulling parts from dates (year, month, day), AGE — calculating time differences, TO_CHAR — formatting dates as readable text, DATE_TRUNC — truncating to period boundaries, Interval arithmetic — adding/subtracting time, 4 practice problems (Easy → Crazy)
Practice Drill: 6 Curriculum Problems	15 mins	Proper-case names, phone masking, email domains, Date formatting, interval math, DATE_TRUNC
Summary & Wrap-up	10 mins	Interview bank — function questions, Syntax Reference Card, Homework + Conditional Logic preview

YOUR SQL JOURNEY



SQL Intro



Setup & RetailMart



DDL & Data Types



Constraints & DML



Filtering & Sorting



Scalar Functions (String & Date)

← HERE



Conditional Logic



Aggregation

5-Minute Memory Check

SQL categories — DDL, DML, DQL, DCL, TCL.

RetailMart V3 — 16 schemas, 55 tables, ~2.7M rows.

DDL + Data Types — CREATE, ALTER, DROP, INT, VARCHAR, NUMERIC.

Constraints + DML — PK, FK, CHECK, INSERT, UPDATE, DELETE.

DQL — SELECT, WHERE, AND/OR/NOT, LIKE, BETWEEN, IN, IS NULL, ORDER BY, LIMIT.

Today: We can FIND data. Now we learn to TRANSFORM it — clean names, format dates, calculate ages, extract parts from strings.

The Wedding Invitation Disaster

Your cousin's wedding. You are printing 500 invitations. The guest list from the database is a MESS:

- Names: 'rAhUL sHaRMa', 'PRIYA PATEL', 'amit kumar'
- Phones: ' 9876 543 210 ' (random spaces, dashes)
- Dates: '1990-04-15T00:00:00+05:30' instead of '15 April 1990'
- Emails: need to extract domain to know who uses gmail vs outlook

The printer needs PERFECTLY formatted data. You need functions that transform messy values into clean output — without changing the original data.

From Story to SQL Functions!

Fix names → `UPPER()`, `LOWER()`, `INITCAP()`

Clean phone numbers → `TRIM()`, `REPLACE()`

Format dates → `TO_CHAR(date, 'DD Month YYYY')`

Calculate age → `AGE(birth_date)`

Extract email domain → `SPLIT_PART(email, '@', 2)`

Extract birth month → `EXTRACT(MONTH FROM date)`

Combine name parts → `CONCAT()` or `||`

These are 'scalar functions' — they work on ONE value and return ONE value. Unlike aggregate functions (Aggregates & Grouping) that work on groups of rows.

Why String Functions Matter

Data analysts spend 80% of their time CLEANING data. Names in wrong case, extra spaces in phone numbers, emails to split into username and domain, product names to standardize. String functions are your data cleaning toolkit.

Key insight: String functions transform the OUTPUT of your query. They do NOT change the stored data. The original table remains untouched.

What Are Scalar Functions?

Technical:

A scalar function takes ONE input value and returns ONE output value. It processes each row independently.

UPPER('hello') always returns 'HELLO' regardless of other rows. This is different from aggregate functions (Aggregates & Grouping) like COUNT/SUM that process multiple rows.

Layman's:

Think of scalar functions like a machine: put ONE item in, get ONE item out. Feed it 'rahul' → it gives you 'RAHUL'. Feed it a date → it gives you the year. It works on ONE cell at a time, row by row.

SYNTAX: Case Conversion

```
-- UPPER: convert to UPPERCASE
SELECT UPPER('hello world'); -- 'HELLO WORLD'

-- LOWER: convert to lowercase
SELECT LOWER('HELLO WORLD'); -- 'hello world'

-- INITCAP: Title Case (First Letter Capital)
SELECT INITCAP('rAhUL sHaRMa'); -- 'Rahul Sharma'
```

LIVE DEMO: Case on RetailMart Customers

-- Run this on your RetailMart database:

```
SELECT
    customer_id,
    first_name,
    UPPER(first_name) AS name_upper,
    LOWER(first_name) AS name_lower,
    INITCAP(first_name) AS name_proper
FROM customers.customers
LIMIT 5;
```

-- Combine first + last name:

```
SELECT
    INITCAP(first_name) || ' ' ||
    INITCAP(last_name) AS full_name
FROM customers.customers LIMIT 5;
```

PRO TIP

PRO TIP

INITCAP is the most useful in real projects. Customer data from web forms is often in random case. `INITCAP('rAhUL sHaRMa')` → 'Rahul Sharma' instantly. Works on multi-word strings too — capitalizes the first letter of EVERY word.

SYNTAX: LENGTH

```
-- LENGTH: count characters in a string
SELECT LENGTH('Hello');           -- 5
SELECT LENGTH('Hello World');    -- 11 (space counts)
SELECT LENGTH('');                -- 0
SELECT LENGTH(NULL);              -- NULL

-- Use in queries – find long product names:
SELECT product_name,
       LENGTH(product_name) AS name_len
FROM products.products
WHERE LENGTH(product_name) > 30
ORDER BY name_len DESC
LIMIT 10;
```

SYNTAX: String Concatenation

```
-- CONCAT: join strings (NULL-safe)
SELECT CONCAT('Hello', ' ', 'World');
    -- 'Hello World'
SELECT CONCAT('Hello', NULL, 'World');
    -- 'HelloWorld' (NULL treated as empty!)

-- || operator: join strings (NULL propagates!)
SELECT 'Hello' || ' ' || 'World';
    -- 'Hello World'
SELECT 'Hello' || NULL || 'World';
    -- NULL! (any NULL → entire result NULL!)

-- On table data:
SELECT CONCAT(first_name, ' ', last_name)
    AS full_name
FROM customers.customers LIMIT 5;
```

CONCAT vs || with NULL

The Mistake:

'Rahul' || NULL || '@gmail.com' = NULL! The || operator propagates NULL — if ANY part is NULL, the entire result is NULL.

The Fix:

Use CONCAT() instead — it treats NULL as empty string: CONCAT('Rahul', NULL, '@gmail.com') = 'Rahul@gmail.com'. In Conditional Logic you will learn COALESCE which provides even more control over NULL handling.

SYNTAX: String Extraction

```
-- SUBSTRING: extract part of text
SELECT SUBSTRING('Hello World' FROM 1 FOR 5);
    -- 'Hello' (start at position 1, take 5 chars)
SELECT SUBSTRING('Hello World' FROM 7);
    -- 'World' (start at 7, take rest)

-- POSITION: find location of substring
SELECT POSITION('@' IN 'user@gmail.com'); -- 5
SELECT POSITION('xyz' IN 'user@gmail.com'); -- 0
    -- Returns 0 if not found (not an error!)

-- SPLIT_PART: split by delimiter, get Nth part
SELECT SPLIT_PART('user@gmail.com', '@', 1);
    -- 'user' (1st part before @)
SELECT SPLIT_PART('user@gmail.com', '@', 2);
    -- 'gmail.com' (2nd part after @)

-- LEFT / RIGHT: shortcut extractors
SELECT LEFT('Hello World', 5); -- 'Hello'
SELECT RIGHT('Hello World', 5); -- 'World'
```


LIVE DEMO: Extract Email Domain

```
-- Two ways to extract domain from email:
```

```
-- Way 1: SPLIT_PART (recommended – clean)
```

```
SELECT email,  
       SPLIT_PART(email, '@', 2) AS domain  
FROM customers.customers LIMIT 5;
```

```
-- Way 2: SUBSTRING + POSITION (manual)
```

```
SELECT email,  
       SUBSTRING(email FROM  
                POSITION('@' IN email) + 1) AS domain  
FROM customers.customers LIMIT 5;
```

```
-- Way 1 is cleaner. Way 2 shows how
```

```
-- POSITION and SUBSTRING work together.
```

SYNTAX: TRIM and REPLACE

```
-- TRIM: remove leading/trailing spaces
SELECT TRIM('  hello  ');          -- 'hello'
SELECT LTRIM('  hello');           -- 'hello' (left only)
SELECT RTRIM('hello  ');           -- 'hello' (right only)
SELECT TRIM(BOTH 'x' FROM 'xxhelloxx');
    -- 'hello' (remove specific char)

-- REPLACE: find and replace text
SELECT REPLACE('Hello World', 'World', 'SQL');
    -- 'Hello SQL'
SELECT REPLACE('Hello World', ' ', '_');
    -- 'Hello_World'

-- Chain REPLACE for multiple cleanups:
SELECT REPLACE(REPLACE(' 98-76 ', '-', ''),
    ' ', ''); -- '9876'
```

PRO TIP

REPLACE replaces ALL occurrences, not just the first one. REPLACE('aaa', 'a', 'b') returns 'bbb'. There is no 'replace first only' in standard SQL — for that you would need REGEXP_REPLACE (advanced). Chain REPLACE calls for multi-step cleaning: REPLACE(REPLACE(phone, ' ', ''), '-', '').

Easy: Customer Name Formatting

EASY

SCENARIO: The marketing team needs customer names in 'Rahul Sharma' format for a banner campaign. The database might have names in mixed case.

TASK: Display `customer_id`, `first_name` in UPPER, LOWER, and INITCAP. Also create a `full_name` column using INITCAP and `||`. Show the cleaned email in lowercase. Display first 10 customers.

HINT: Three string operations stack here — proper-casing each name, then joining the two parts with a space. Trap: `||` returns NULL if any operand is NULL; one missing `last_name` silently produces a NULL full name unless you wrap with COALESCE.

Answer

✓ ANSWER

```
SELECT
  customer_id,
  first_name,
  UPPER(first_name) AS name_upper,
  LOWER(first_name) AS name_lower,
  INITCAP(first_name) || ' ' ||
    INITCAP(last_name) AS full_name,
  LOWER(email) AS email_clean
FROM customers.customers
LIMIT 10;
```

Medium: Email Domain Extraction

MEDIUM

SCENARIO: The marketing team wants to see which email providers customers use (gmail.com, yahoo.com, outlook.com). Extract the domain from each email.

TASK: Show customer_id, email, the domain part (after @), and the username part (before @). Also show the position of @ in each email. Display 15 customers.

HINT: 'Extract domain from email' is just 'give me the text after @' — there's a string function that splits on a delimiter and picks a piece. Trap: SUBSTRING with POSITION works too, but it's verbose and easy to get the off-by-one wrong; the delimiter-aware function is cleaner.

NOTE: In Aggregates & Grouping you will learn GROUP BY + COUNT to count how many customers use each domain. For now, focus on the extraction technique per row.

Answer



```
SELECT
    customer_id,
    email,
    SPLIT_PART(email, '@', 1) AS username,
    SPLIT_PART(email, '@', 2) AS domain,
    POSITION('@' IN email) AS at_position
FROM customers.customers
ORDER BY domain
LIMIT 15;
```

-- Alternative using SUBSTRING + POSITION:

```
SELECT
    email,
    SUBSTRING(email FROM 1 FOR
        POSITION('@' IN email) - 1) AS username,
    SUBSTRING(email FROM
        POSITION('@' IN email) + 1) AS domain
FROM customers.customers LIMIT 10;
```

Hard: Web Session URL Parsing

HARD

SCENARIO: The web analytics team wants to break page-view URLs into parts. RetailMart URLs are relative paths: simple pages like '/home' or '/cart', and product pages like '/product/79' that carry an item id. Extract the SECTION (first folder) and the ITEM ID (second folder — only product pages have one).

TASK: From `web_events.page_views`, extract: `section` (SPLIT_PART on '/', part 2), `item_id` (part 3 — turn the empty string into a real NULL with NULLIF), `device_type` lowercased. Filter to mobile/tablet only. Run it twice: shortest URLs first (`item_id` is NULL), then product pages only (`item_id` populated).

HINT: '/product/79' splits on '/' into parts: '' (before the first slash), 'product', '79' — so `section` = part 2, `id` = part 3. Trap: when a part does not exist, SPLIT_PART returns an EMPTY STRING, not NULL — wrap it in NULLIF(x, '') to get a true NULL you can filter and count later.

Answer (1/2)

✓ ANSWER

-- Simple pages first: item_id comes out NULL (thanks to NULLIF)

```
SELECT
  view_id,
  page_url,
  SPLIT_PART(page_url, '/', 2) AS section,
  NULLIF(SPLIT_PART(page_url, '/', 3), '')
    AS item_id,
  LOWER(device_type) AS device
FROM web_events.page_views
WHERE LOWER(device_type)
      IN ('mobile', 'tablet')
ORDER BY LENGTH(page_url)
LIMIT 8;
```

-- Product pages: item_id is populated

```
SELECT
  view_id,
  page_url,
  SPLIT_PART(page_url, '/', 2) AS section,
  NULLIF(SPLIT_PART(page_url, '/', 3), '')
    AS item_id
FROM web_events.page_views
```

Answer (2/2)

✓ ANSWER

```
WHERE page_url LIKE '/product/%'  
LIMIT 7;
```

Crazy: Product Catalog Display - Scalar Showcase

CRAZY

SCENARIO: The catalog team wants a clean product display drawn from ONE table (products.products), one row at a time: a proper-case trimmed name, the price as Indian currency, and the profit margin percent per product. No grouping, no joins - pure per-row scalar functions (those come on later days).

TASK: Combine string functions (INITCAP, TRIM), number formatting (TO_CHAR), arithmetic (margin = (price - cost_price) / price), and a NULLIF guard. Show 15 rows ordered by price descending.

HINT: Margin involves division — and division by zero crashes. Trap: dividing by price breaks if any price is 0; wrap the denominator with NULLIF(price, 0) so the result becomes NULL instead of an error.

Answer

✓ ANSWER

```
SELECT
    product_id,
    INITCAP(TRIM(product_name)) AS clean_name,
    'Rs. ' || TO_CHAR(price, 'FM99,99,99,999.00')
        AS price_fmt,
    ROUND(100.0 * (price - cost_price)
        / NULLIF(price, 0), 1) || '%'
        AS margin_pct
FROM products.products
ORDER BY price DESC
LIMIT 15;
```

Try It Live!

Run each query on your RetailMart database. Modify them. Break them. Understanding comes from experimentation. These take 1-2 minutes each.

Challenge 1: Reverse Name Format

```
-- Create 'SHARMA R.' format from customer names
-- (last name uppercase + first initial + dot)
SELECT
    UPPER(last_name) || ' ' ||
        UPPER(LEFT(first_name, 1)) || '.'
    AS short_name
FROM customers.customers LIMIT 10;

-- YOUR TURN: Try 'R. Sharma' format instead
-- (first initial + dot + space + initcap last)
```

Challenge 2: Email Username Length

```
-- Find customers with long email usernames
SELECT email,
       SPLIT_PART(email, '@', 1) AS username,
       LENGTH(SPLIT_PART(email, '@', 1))
         AS username_len
FROM customers.customers
WHERE LENGTH(SPLIT_PART(email, '@', 1)) > 15
ORDER BY username_len DESC
LIMIT 10;

-- YOUR TURN: Find emails with short usernames
-- (less than 5 characters before @)
```

Challenge 3: Product Name Word Count

```
-- Estimate word count using spaces
-- (length of name - length without spaces + 1)
SELECT
    product_name,
    LENGTH(product_name) -
        LENGTH(REPLACE(product_name, ' ', ''))
        + 1 AS word_count_estimate
FROM products.products
ORDER BY word_count_estimate DESC
LIMIT 10;

-- YOUR TURN: Find products with exactly
-- 1-word names (no spaces in name)
```


Challenge 4: Store Name Initials

```
-- Build store initials from store_name
SELECT
    store_id,
    store_name,
    UPPER(LEFT(store_name, 1)) ||
        UPPER(LEFT(SPLIT_PART(store_name,
            ' ', 2), 1)) AS store_initials
FROM stores.stores
LIMIT 10;

-- YOUR TURN: Show store names in UPPER
-- with the city in parentheses
-- e.g., 'MEGA MART (Mumbai)'
```

KEY TAKEAWAYS

UPPER/LOWER/INITCAP for case conversion. LENGTH for character counting. CONCAT (NULL-safe) or || (NULL propagates) for joining. SUBSTRING for extracting. SPLIT_PART for delimiter-based splitting. TRIM for whitespace. REPLACE for find-and-replace. POSITION for locating substrings. LEFT/RIGHT for quick slicing.

Golden Rule: These functions transform the OUTPUT only — the stored data in the table never changes.

Extract Domain from Email

Q: 'How do you extract the domain from an email address in SQL?'

A: Two approaches:

1) `SPLIT_PART(email, '@', 2)` — splits by @ and returns the 2nd part. Clean and readable.

2) `SUBSTRING(email FROM POSITION('@' IN email) + 1)` — finds @ position, takes everything after.

`SPLIT_PART` is preferred for readability and simplicity.

Why Date Functions Matter

Every business question involves TIME: 'When was this order placed?' 'How long has this customer been with us?' 'Show orders from Q2 2025.' 'Format this date for the invoice.'

Date functions let you extract parts of dates, calculate time differences, format dates for display, and truncate to period boundaries.

Date/Time Types in PostgreSQL

Technical:

PostgreSQL has 4 main temporal types: DATE (date only, '2025-05-18'), TIMESTAMP (date + time, '2025-05-18 14:30:00'), TIMESTAMPTZ (with timezone, '2025-05-18 14:30:00+05:30'), INTERVAL (duration, '3 years 2 months'). Most RetailMart columns use TIMESTAMPTZ or DATE.

Layman's:

DATE = calendar date. TIMESTAMP = date + clock time. INTERVAL = a duration (like '5 days' or '2 years'). Functions convert between these and pull out parts.

SYNTAX: Current Date/Time (1/2)

```
-- Current timestamp (with time + timezone)
SELECT NOW();
    -- 2025-05-21 14:30:00.123456+05:30

-- Current date only (no time)
SELECT CURRENT_DATE;    -- 2025-05-21

-- Current time only
SELECT CURRENT_TIME;    -- 14:30:00.123456+05:30

-- CURRENT_TIMESTAMP = same as NOW()
SELECT CURRENT_TIMESTAMP;

-- Use in WHERE: orders from the dataset's LAST day.
-- (RetailMart data runs 2024-01-01 to 2026-02-26, so
--  '>= CURRENT_DATE' would return nothing - in production
--  with live data, CURRENT_DATE is exactly what you'd use.)
SELECT order_id, order_date
FROM sales.orders
WHERE order_date >= DATE '2026-02-26'
```

SYNTAX: Current Date/Time (2/2)

```
LIMIT 5;
```

SYNTAX: EXTRACT

```
-- EXTRACT(part FROM date) – pull components
SELECT EXTRACT(YEAR FROM NOW());      -- 2025
SELECT EXTRACT(MONTH FROM NOW());     -- 5
SELECT EXTRACT(DAY FROM NOW());       -- 21
SELECT EXTRACT(HOUR FROM NOW());      -- 14
SELECT EXTRACT(DOW FROM NOW());       -- 0-6 (Sun-Sat)
SELECT EXTRACT(QUARTER FROM NOW());   -- 2
SELECT EXTRACT(EPOCH FROM NOW());
    -- seconds since 1970-01-01 (Unix time)

-- Use with table columns:
SELECT order_id,
    EXTRACT(YEAR FROM order_date) AS order_year,
    EXTRACT(MONTH FROM order_date) AS order_month,
    EXTRACT(DOW FROM order_date) AS day_of_week
FROM sales.orders LIMIT 10;
```


PRO TIP

EXTRACT(DOW FROM date) returns: 0=Sunday, 1=Monday ... 6=Saturday. This is useful for finding weekend orders.

EXTRACT(ISODOW FROM date) returns 1=Monday ... 7=Sunday (ISO standard). EXTRACT(QUARTER FROM date) returns 1-4. All these return a number you can use in WHERE.

SYNTAX: AGE and Intervals

```
-- AGE with one argument: time since that date
SELECT AGE(TIMESTAMP '2020-06-15');
    -- '4 years 11 mons 6 days' (from NOW)

-- AGE with two arguments: difference
SELECT AGE('2025-01-01', '2020-06-15');
    -- '4 years 6 mons 16 days'

-- Interval arithmetic – add/subtract time:
SELECT NOW() - INTERVAL '30 days';    -- 30 days ago
SELECT NOW() + INTERVAL '1 year';    -- 1 year later
SELECT NOW() + INTERVAL '2 hours';   -- 2 hours later

-- On table data:
SELECT order_id,
       order_date,
       AGE(order_date) AS order_age
FROM sales.orders
ORDER BY order_date LIMIT 5;
```

LIVE DEMO: Interval Arithmetic on Orders (1/2)

```
-- Our data's 'today' = its last order day, 2026-02-26.  
-- (With live data you'd write NOW() - the INTERVAL math is identical.)
```

```
-- Orders in the data's last 30 days:  
SELECT order_id, order_date, net_total  
FROM sales.orders  
WHERE order_date >= DATE '2026-02-26' - INTERVAL '30 days'  
ORDER BY order_date DESC  
LIMIT 10;
```

```
-- Orders in the data's last 7 days:  
SELECT order_id, order_date  
FROM sales.orders  
WHERE order_date >= DATE '2026-02-26' - INTERVAL '7 days'  
LIMIT 10;
```

```
-- Estimated delivery (order + 7 days):  
SELECT order_id, order_date,  
       order_date + INTERVAL '7 days'  
       AS estimated_delivery
```

LIVE DEMO: Interval Arithmetic on Orders (2/2)

```
FROM sales.orders LIMIT 5;
```

SYNTAX: TO_CHAR

```
-- TO_CHAR(date, format_string)
SELECT TO_CHAR(NOW(), 'DD-Mon-YYYY');
      -- '21-May-2025'
SELECT TO_CHAR(NOW(), 'Day, DD Month YYYY');
      -- 'Wednesday, 21 May 2025'
SELECT TO_CHAR(NOW(), 'HH12:MI AM');
      -- '02:30 PM'
SELECT TO_CHAR(NOW(), 'YYYY/MM/DD');
      -- '2025/05/21'

-- On table data:
SELECT order_id,
       TO_CHAR(order_date, 'DD-Mon-YYYY')
       AS formatted
FROM sales.orders LIMIT 5;
```

TO_CHAR Format Codes

YYYY = 4-digit year (2025). YY = 2-digit year (25).

MM = month number (01-12). Mon = abbreviated (Jan, May). Month = full name (January, May).

DD = day of month (01-31). Day = weekday name (Monday). Dy = short weekday (Mon).

HH12 = 12-hour clock. HH24 = 24-hour clock. MI = minutes. SS = seconds. AM/PM = meridiem.

Q = quarter number (1-4).

Example: TO_CHAR(date, 'DD-Mon-YYYY (Day)') → '21-May-2025 (Wednesday)'

SYNTAX: DATE_TRUNC

```
-- DATE_TRUNC('precision', date)
-- Rounds DOWN to the start of the period

SELECT DATE_TRUNC('month', NOW());
-- 2025-05-01 00:00:00 (start of month)
SELECT DATE_TRUNC('year', NOW());
-- 2025-01-01 00:00:00 (start of year)
SELECT DATE_TRUNC('quarter', NOW());
-- 2025-04-01 (start of Q2)
SELECT DATE_TRUNC('week', NOW());
-- start of current week (Monday)
SELECT DATE_TRUNC('day', NOW());
-- 2025-05-21 00:00:00 (midnight today)
```

LIVE DEMO: DATE_TRUNC on Orders

```
-- Show each order with its month boundary:
SELECT order_id,
       order_date,
       DATE_TRUNC('month', order_date)
       AS month_start,
       TO_CHAR(DATE_TRUNC('month', order_date),
               'Mon YYYY') AS month_label
FROM sales.orders
WHERE order_date >= '2025-01-01'
ORDER BY order_date
LIMIT 15;

-- Notice: ALL January orders show
-- month_start = '2025-01-01'
-- In Aggregates & Grouping, you will GROUP BY month_start
-- to get monthly totals!
```


PRO TIP

PRO TIP

DATE_TRUNC is the foundation of all time-based reporting. In Aggregates & Grouping, you will combine DATE_TRUNC with GROUP BY and SUM to create monthly/quarterly/yearly revenue summaries. For now, understand what DATE_TRUNC does to individual dates — it rounds them DOWN to the start of the period.

Easy: Order Date Formatting

EASY

SCENARIO: The sales report needs order dates in Indian format: '15-Jan-2025' instead of raw '2025-01-15 14:30:00+05:30'. Also show which year, month, and quarter each order belongs to.

TASK: Show order_id, original order_date, formatted date using TO_CHAR ('DD-Mon-YYYY'), day name, extracted year, month, and quarter. Show 10 orders.

HINT: Two complementary date tools — one formats a date for DISPLAY (output is text), the other pulls a numeric PART out (output is a number). Trap: don't confuse them — using TO_CHAR for math, or EXTRACT for a display column, leads to wrong types later in the pipeline.

Answer

✓ ANSWER

```
SELECT
  order_id,
  order_date,
  TO_CHAR(order_date, 'DD-Mon-YYYY')
    AS formatted_date,
  TO_CHAR(order_date, 'Day')
    AS day_name,
  EXTRACT(YEAR FROM order_date) AS yr,
  EXTRACT(MONTH FROM order_date) AS mon,
  EXTRACT(QUARTER FROM order_date) AS qtr
FROM sales.orders
LIMIT 10;
```

Medium: Days Since Order — Interval Math

MEDIUM

SCENARIO: The retention team wants to find OLD orders — those placed more than 90 days ago. For each order, calculate how many days ago it was placed.

TASK: Show `order_id`, `order_date`, and calculate the age using `AGE()`. Also calculate total days using `EXTRACT(EPOCH)`. Filter to orders older than 90 days. Show oldest first.

HINT: Two ways to express 'days since X' — `AGE` returns a HUMAN interval (years/months/days), and pure date subtraction returns a NUMBER (days). Trap: subtracting timestamps gives an interval, not days — wrap with `EXTRACT(EPOCH FROM ...)` / 86400 if you need a plain day count for sorting or comparisons.

WHY 86400? There are 86,400 seconds in a day (24 x 60 x 60). `EXTRACT(EPOCH)` gives seconds, so dividing by 86400 converts to days. `ROUND` removes decimals.

Answer

✓ ANSWER

-- Step 1: See order age as interval

```
SELECT order_id, order_date,  
       AGE(order_date) AS order_age  
FROM sales.orders  
ORDER BY order_date  
LIMIT 10;
```

-- Step 2: Get total days (numeric)

```
SELECT order_id, order_date,  
       ROUND(EXTRACT(EPOCH FROM  
                   NOW() - order_date) / 86400)  
       AS days_ago  
FROM sales.orders  
WHERE EXTRACT(EPOCH FROM  
              NOW() - order_date) / 86400 > 90  
ORDER BY order_date  
LIMIT 15;
```

Hard: DATE_TRUNC — Monthly Order View

HARD

SCENARIO: The finance team wants to see orders organized by month for 2025. Instead of raw dates, they want each order labeled with its month.

TASK: Show order_id, order_date, the month start (DATE_TRUNC), a formatted month label (TO_CHAR), and the net_total. Filter to 2025 orders only. Order by date.

HINT: 'Bucket by month' means dropping the day part — there's a date function that rounds to a unit boundary (month, week, day). Trap: DATE_TRUNC keeps the year and month but RESETS day to the 1st with time 00:00:00; if you need a human label like 'Mar 2024', layer TO_CHAR on top.

Answer

✓ ANSWER

```
SELECT
  order_id,
  order_date,
  DATE_TRUNC('month', order_date)
    AS month_start,
  TO_CHAR(DATE_TRUNC('month', order_date),
    'Mon YYYY') AS month_label,
  net_total
FROM sales.orders
WHERE order_date >= '2025-01-01'
  AND order_date < '2026-01-01'
ORDER BY order_date
LIMIT 20;

-- Notice: all January orders show
-- month_label = 'Jan 2025'
-- Aggregates & Grouping preview: GROUP BY month_start
-- + SUM(net_total) = monthly revenue!
```

Crazy: HR Attendance — Working Hours Calculation

CRAZY

SCENARIO: The HR Head needs a daily working-hours report. From hr.attendance, calculate each employee's hours worked (check_out - check_in), day of week, and flag short days (under 6 hours). Combine timestamps, intervals, and formatting.

TASK: From hr.attendance, show employee_id, formatted check-in date, day name, hours worked (numeric, 1 decimal), and a flag column showing 'Short Day' if < 6 hours else 'Normal'. Latest 20 records.

HINT: Subtracting two timestamps gives an INTERVAL — to convert to numeric hours you need EXTRACT(EPOCH) / 3600.

Trap: rows where check_out IS NULL (open shifts) will give NULL hours — exclude them in WHERE, otherwise the 'Short Day' flag silently misclassifies them.

Answer

✓ ANSWER

```
SELECT
    employee_id,
    TO_CHAR(attendance_date,
        'DD-Mon-YYYY') AS work_date,
    TRIM(TO_CHAR(attendance_date,
        'FMDay')) AS day_name,
    ROUND(EXTRACT(EPOCH FROM
        check_out - check_in) / 3600.0, 1)
        AS hours_worked,
    CASE
        WHEN EXTRACT(EPOCH FROM
            check_out - check_in) / 3600 < 6
            THEN 'Short Day'
        ELSE 'Normal'
    END AS shift_flag
FROM hr.attendance
WHERE check_out IS NOT NULL
ORDER BY attendance_date DESC LIMIT 20;
```

Try It Live!

Run each query. Try changing the intervals, the date parts, the format strings. Understanding comes from experimentation!

Challenge 1: Orders This Quarter

```
-- Find orders in the data's FINAL quarter (its 'current' quarter).
-- Live data version: DATE_TRUNC('quarter', NOW())
SELECT order_id, order_date, net_total
FROM sales.orders
WHERE order_date >= DATE_TRUNC('quarter', DATE '2026-02-26')
ORDER BY order_date DESC
LIMIT 10;

-- YOUR TURN: Find orders from LAST quarter
-- Hint: DATE_TRUNC('quarter',
--     NOW() - INTERVAL '3 months')
```

Challenge 2: Weekend Registration Check

```
-- Show which day of the week customers registered
SELECT
    customer_id,
    INITCAP(first_name) AS name,
    TO_CHAR(registration_date, 'Day')
        AS reg_day,
    EXTRACT(DOW FROM registration_date)
        AS dow_num
FROM customers.customers
WHERE EXTRACT(DOW FROM registration_date)
    IN (0, 6)
LIMIT 10;

-- This shows weekend registrations!
-- 0=Sunday, 6=Saturday
-- YOUR TURN: Find WEEKDAY registrations
-- (dow NOT IN (0, 6))
```

Challenge 3: Future Date Calculation

-- When does each employee complete 5 years?

```
SELECT
    employee_id,
    INITCAP(first_name) AS name,
    joining_date,
    joining_date + INTERVAL '5 years'
        AS five_year_anniversary,
    TO_CHAR(joining_date + INTERVAL '5 years',
        'DD-Mon-YYYY') AS anniversary_fmt
FROM stores.employees
ORDER BY joining_date
LIMIT 10;
```

-- YOUR TURN: Find employees whose

-- 5-year anniversary is in 2025

Challenge 4: Multiple TO_CHAR Formats

```
-- Same date, different display formats:
SELECT
  order_id,
  order_date,
  TO_CHAR(order_date, 'DD-Mon-YYYY')
    AS format_1,
  TO_CHAR(order_date, 'YYYY/MM/DD')
    AS format_2,
  TO_CHAR(order_date, 'Day')
    AS format_3,
  TO_CHAR(order_date, 'Month DD, YYYY')
    AS format_4,
  TO_CHAR(order_date, 'DD/MM/YY HH12:MI AM')
    AS format_5
FROM sales.orders LIMIT 5;

-- YOUR TURN: Create your own format!
-- Try: 'Dy, DD Mon YYYY' or 'FMMonth FMDD'
```

KEY TAKEAWAYS

NOW() = current timestamp. CURRENT_DATE = today. EXTRACT(part FROM date) = pull year/month/day/quarter/dow. AGE(date) = time since that date as interval. TO_CHAR(date, format) = formatted display. DATE_TRUNC('period', date) = round to period start. Interval math: date +/- INTERVAL '30 days'.

Key pattern: DATE_TRUNC + TO_CHAR for time-based labeling. EXTRACT(EPOCH ...) / 86400 for numeric day count.

EXTRACT vs DATE_PART

The Mistake:

Using `DATE_PART('month', date)` instead of `EXTRACT(MONTH FROM date)`. Both work, but they have subtle differences in older PostgreSQL versions.

The Fix:

Use `EXTRACT` — it is the SQL standard. `DATE_PART` is PostgreSQL-specific. In modern PostgreSQL (14+) both return numeric, but `EXTRACT` is more portable and recommended by official documentation.

Date Arithmetic in Business Queries

Q: 'How do you find records from the last 30 days?'

A: WHERE order_date >= NOW() - INTERVAL '30 days'. Use >= with INTERVAL, not BETWEEN. For the previous full calendar month: WHERE order_date >= DATE_TRUNC('month', NOW() - INTERVAL '1 month') AND order_date < DATE_TRUNC('month', NOW()).

Why This Section Matters

These 6 patterns appear in every data analyst's daily work. Master them and you will clean, format, and transform data like a professional. Each one uses **ONLY** functions learned today — no GROUP BY, no CASE WHEN, no JOINS.

Proper-Case Names

```
-- Display customer names in 'Sharma, Rahul' format
SELECT
  customer_id,
  INITCAP(last_name) || ', ' ||
    INITCAP(first_name) AS display_name,
  UPPER(LEFT(first_name, 1)) || '.' ||
    UPPER(LEFT(last_name, 1)) || '.'
    AS initials
FROM customers.customers
LIMIT 10;
```

Phone Masking

```
-- Show only last 4 digits of phone
SELECT
    customer_id,
    INITCAP(first_name) AS name,
    'XXXXXX' || RIGHT(phone, 4)
        AS masked_phone
FROM customers.customers
WHERE phone IS NOT NULL
LIMIT 10;
```

Email Domain Extraction

```
-- Show domain and username from email
SELECT
    customer_id,
    email,
    SPLIT_PART(email, '@', 1) AS username,
    SPLIT_PART(email, '@', 2) AS domain
FROM customers.customers
WHERE email LIKE '%gmail%'
    OR email LIKE '%yahoo%'
LIMIT 10;
```

Monthly Order Labels

```
-- Label each order with its month
SELECT
  order_id,
  order_date,
  DATE_TRUNC('month', order_date)
    AS month_boundary,
  TO_CHAR(order_date, 'Mon YYYY')
    AS month_label,
  'Q' || EXTRACT(QUARTER FROM order_date)
    AS quarter_label
FROM sales.orders
WHERE order_date >= '2025-01-01'
ORDER BY order_date
LIMIT 15;
```

Days Since Order

```
-- How many days ago was each order placed?
SELECT
  order_id,
  order_date,
  AGE(order_date) AS time_ago,
  ROUND(EXTRACT(EPOCH FROM
    NOW() - order_date) / 86400)
    AS days_ago
FROM sales.orders
ORDER BY order_date DESC
LIMIT 10;
```

Date Formatting Examples (1/2)

```
-- Format support ticket dates:
SELECT
    ticket_id,
    subject,
    created_date,
    TO_CHAR(created_date, 'DD-Mon-YYYY')
        AS formatted_date,
    TO_CHAR(created_date, 'Day, DD Month YYYY')
        AS full_format
FROM support.tickets LIMIT 10;
```

```
-- Multiple format styles on orders:
SELECT
    order_id,
    TO_CHAR(order_date, 'DD-Mon-YYYY')
        AS indian_format,
    TO_CHAR(order_date, 'YYYY/MM/DD')
        AS iso_format,
    TO_CHAR(order_date, 'Day, DD Month YYYY')
        AS full_format,
```


Date Formatting Examples (2/2)

```
TO_CHAR(order_date, 'DD/MM/YY')  
  AS short_format  
FROM sales.orders LIMIT 10;
```

LENGTH on NULL

The Mistake:

WHERE LENGTH(phone) = 10 — if phone is NULL, LENGTH(NULL) returns NULL, and NULL = 10 is UNKNOWN (not TRUE or FALSE). These rows are silently skipped.

The Fix:

Always add WHERE phone IS NOT NULL before checking LENGTH. Or: WHERE phone IS NOT NULL AND LENGTH(phone) = 10.

SUBSTRING Position is 1-based

The Mistake:

SUBSTRING('Hello' FROM 0 FOR 3) — you might expect 'Hel' but PostgreSQL uses 1-based indexing. Position 0 is treated as position 1 with count reduced by 1, giving 'He' instead!

The Fix:

Always start from position 1: SUBSTRING('Hello' FROM 1 FOR 3) → 'Hel'. Unlike most programming languages, SQL strings are 1-indexed.

TO_CHAR Month Padding

The Mistake:

TO_CHAR(date, 'Month') returns 'May ' (padded with spaces to 9 characters). When you concatenate: 'Month: ' || TO_CHAR(date, 'Month') || ' 2025' → 'Month: May 2025'.

The Fix:

Use FM prefix: TO_CHAR(date, 'FMMonth') → 'May' (no padding). FM = Fill Mode, removes padding from Month, Day, etc.

INTERVAL vs Integer Math

The Mistake:

`order_date + 7` — adding an integer to a date works for DATE type (adds days), but fails or gives wrong results with TIMESTAMP. Inconsistent behavior.

The Fix:

Always use INTERVAL: `order_date + INTERVAL '7 days'`. This works correctly with both DATE and TIMESTAMP types. Explicit is better than implicit.

The Data Analyst's Daily Routine

In a real company, you receive messy data from multiple sources: web forms, CSV imports, API integrations. Your job is to clean it for reports. Let us simulate that workflow using RetailMart.

Customer Data Quality (1/2)

-- Check: Do any customers have very short names? (3 letters or fewer)

```
SELECT customer_id, first_name, last_name,  
       LENGTH(first_name) AS fn_len,  
       LENGTH(last_name) AS ln_len  
FROM customers.customers  
WHERE LENGTH(first_name) <= 3  
       OR LENGTH(last_name) <= 3  
LIMIT 10;
```

-- 3-letter names (Raj, Dev, Om...) are legit - the check

-- SURFACES them so a human can judge. That is auditing.

-- Check: is phone ever missing? count(*) vs count(column)

```
SELECT count(*) AS customers,  
       count(phone) AS with_phone  
FROM customers.customers;
```

-- Equal! -> the signup system ENFORCES phone. A clean pass

-- is also a finding - now you can trust this column.

-- A REAL gap lives in support: tickets never resolved

```
SELECT ticket_id,
```

Customer Data Quality (2/2)

```
INITCAP(category) AS category,  
status, created_date  
FROM support.tickets  
WHERE resolved_date IS NULL  
LIMIT 10;
```


Order Data Check

```
-- Show orders with their formatted details
SELECT
    order_id,
    TO_CHAR(order_date, 'DD-Mon-YYYY (Dy)')
        AS order_formatted,
    UPPER(order_status) AS status,
    net_total,
    AGE(order_date) AS how_long_ago
FROM sales.orders
WHERE order_status = 'Processing'
ORDER BY order_date
LIMIT 15;

-- The oldest still-Processing orders might need attention!
```

Clean Employee Directory

```
-- Build a presentation-ready employee list
SELECT
  'EMP-' || LPAD(employee_id::text, 5, '0')
    AS emp_code,
  INITCAP(first_name) || ' ' ||
    INITCAP(last_name) AS full_name,
  UPPER(role) AS designation,
  TO_CHAR(joining_date, 'FMMonth YYYY')
    AS joined_in,
  EXTRACT(YEAR FROM AGE(joining_date))
    || '+yrs' AS experience
FROM stores.employees
ORDER BY joining_date
LIMIT 15;
```

Clean Product Catalog

```
-- Build a formatted product listing
SELECT
  'SKU-' || product_id AS sku,
  INITCAP(product_name) AS name,
  'Rs. ' || price AS price_display,
  'Rs. ' || cost_price AS cost_display,
  LENGTH(product_name) AS name_chars,
  SPLIT_PART(product_name, ' ', 1)
    AS brand_guess
FROM products.products
WHERE price > 1000
ORDER BY price DESC
LIMIT 15;
```

Scalar vs Aggregate Functions

Q: 'What is the difference between scalar and aggregate functions?'

A: Scalar functions work on ONE value and return ONE value — `UPPER('hello') → 'HELLO'`. They process each row independently. Aggregate functions work on MANY rows and return ONE value — `COUNT(*) → 50000`. Scalar functions can be used in `SELECT`, `WHERE`, `ORDER BY`. Aggregates need `GROUP BY` (Aggregates & Grouping).

NULL Handling in String Functions

Q: 'How do you handle NULL values in string concatenation?'

A: The `||` operator returns NULL if any part is NULL. `CONCAT()` treats NULL as empty string. So `CONCAT(first_name, ' ', last_name)` is safer than `first_name || ' ' || last_name` when data might be NULL. In Conditional Logic, you will learn `COALESCE(value, 'default')` for explicit NULL replacement.

POSITION vs STRPOS

Q: 'How do you find the position of a character in a string?'

A: SQL standard: POSITION('@' IN email). PostgreSQL-specific: STRPOS(email, '@'). Both return 1-based position. Both return 0 if not found (not an error). Use with SUBSTRING to extract parts: SUBSTRING(email FROM POSITION('@' IN email) + 1) extracts the domain.

DATE_TRUNC for Reporting

Q: 'How would you create a monthly revenue report?'

A: Use DATE_TRUNC('month', order_date) to round all dates to the 1st of their month. Then GROUP BY that value and SUM(net_total) for monthly totals (you will learn this in Aggregates & Grouping). TO_CHAR formats the month label.

This pattern works for any period: 'quarter', 'year', 'week'.

EXTRACT Use Cases

Q: 'When would you use EXTRACT vs DATE_TRUNC?'

A: EXTRACT pulls a single component (just the year, just the month) as a number. DATE_TRUNC returns a full truncated date/timestamp. Use EXTRACT when you need a number for filtering: 'WHERE EXTRACT(MONTH ...) = 5'. Use DATE_TRUNC when you need date boundaries for grouping: 'all orders in the same month as one group'.

TRIM vs REPLACE for Cleaning

Q: 'What is the difference between TRIM and REPLACE?'

A: TRIM removes characters from the START and END of a string only. REPLACE finds and replaces ALL occurrences EVERYWHERE in the string. TRIM(' hello ') removes outer spaces. REPLACE('h e l l o', ' ', '') removes ALL spaces. Use TRIM for edge cleanup, REPLACE for global find-and-replace.

SPLIT_PART vs SUBSTRING

Q: 'When would you use SPLIT_PART vs SUBSTRING?'

A: SPLIT_PART is best when data has a consistent delimiter — email (@), CSV (,), path (/). SUBSTRING is for fixed-position extraction — first 5 characters, characters 3-7. SPLIT_PART('user@gmail.com', '@', 2) is cleaner than SUBSTRING(email FROM POSITION('@' IN email) + 1).

How to Find Records from a Specific Month

Q: 'Find all orders from March 2025.'

A: Two ways: (1) EXTRACT: WHERE EXTRACT(YEAR FROM order_date) = 2025 AND EXTRACT(MONTH FROM order_date) = 3. (2) DATE_TRUNC + range: WHERE order_date >= '2025-03-01' AND order_date < '2025-04-01'. Method 2 is faster because it can use indexes on order_date.

KEY TAKEAWAYS

STRING FUNCTIONS:

1. UPPER / LOWER / INITCAP — case conversion. INITCAP for display names.
2. LENGTH — count characters. Useful for data validation.
3. CONCAT (NULL-safe) or || (NULL propagates) — join strings.
4. SUBSTRING — extract parts. SPLIT_PART for delimiter-based splitting.
5. TRIM — remove whitespace. REPLACE — find and replace text.
6. POSITION — find substring location. LEFT / RIGHT for shortcuts.

DATE FUNCTIONS:

7. NOW() / CURRENT_DATE — current timestamp / date.
8. EXTRACT — pull year / month / day / quarter / dow from dates.
9. AGE — calculate time differences as interval.
10. TO_CHAR — format dates for human-readable display.
11. DATE_TRUNC — round dates to period boundaries.
12. INTERVAL — arithmetic: date +/- INTERVAL '30 days'.

Golden Rule: Scalar functions transform OUTPUT, not stored data. They work on ONE value at a time, row by row.

Scalar Functions (String & Date)

-- Case Conversion

```
UPPER('hello')    → 'HELLO'  
LOWER('HELLO')   → 'hello'  
INITCAP('rAhUL') → 'Rahul'
```

-- String Measurement

```
LENGTH('hello') → 5  
POSITION('@' IN email) → position of @
```

-- String Joining

```
CONCAT(a, ' ', b) → NULL-safe join  
a || ' ' || b     → NULL propagates
```

Scalar Functions (String & Date)

-- String Extraction

```
SUBSTRING(str FROM 1 FOR 5)
SPLIT_PART(email, '@', 2) → domain
LEFT(str, 3) / RIGHT(str, 4)
```

-- String Cleaning

```
TRIM(' hello ') → 'hello'
REPLACE(str, 'old', 'new')
```

-- Date/Time Current

```
NOW()           → timestamp with tz
CURRENT_DATE    → date only
CURRENT_TIMESTAMP → same as NOW()
```

Scalar Functions (String & Date)

-- Date Extraction

```
EXTRACT(YEAR FROM date)
EXTRACT(MONTH FROM date)
EXTRACT(DOW FROM date)    → 0=Sun
EXTRACT(QUARTER FROM date)
```

-- Date Calculation

```
AGE(date)                → interval from now
AGE(date1, date2)        → difference
date + INTERVAL '30 days'
date - INTERVAL '1 month'
```

-- Date Formatting

```
TO_CHAR(date, 'DD-Mon-YYYY')
TO_CHAR(date, 'Month YYYY')
TO_CHAR(date, 'HH12:MI AM')
```

Scalar Functions (String & Date)

-- Date Truncation

```
DATE_TRUNC('month', date)    → 1st of month  
DATE_TRUNC('quarter', date) → 1st of quarter  
DATE_TRUNC('year', date)     → Jan 1st
```


CHALLENGE

CHALLENGE (1/3)

Use scalar functions to build these queries on RetailMart V3:

CHALLENGE (2/3)

- 1.: Display all customer names in 'Sharma, Rahul' format (last, first) using INITCAP and ||.
- 2.: Mask all customer phone numbers — show 'XXXXXX' + last 4 digits only. Filter out NULLs with WHERE.
- 3.: Show each customer's email domain using SPLIT_PART. Filter to gmail.com users only (use LIKE).
- 4.: Show products with names longer than 40 characters. Display name, length, and first word (SPLIT_PART by space).
- 5.: Create a formatted order report: order_id, date as 'DD-Mon-YYYY (Day)', net_total. Show 20 orders from 2025.
- 6.: For each employee, show name, joining_date, and tenure as 'X yrs Y mos' using AGE + EXTRACT.
- 7.: Find orders placed more than 180 days ago. Show order_id, order_date, and total days since. Use EXTRACT(EPOCH) / 86400.
- 8.:

CHALLENGE

CHALLENGE (3/3)

Show each order with its quarter label: 'Q1 2025', 'Q2 2025'. Use `EXTRACT(QUARTER)` and `EXTRACT(YEAR)` with `||`.

Push your SQL file to GitHub!

<https://github.com/starlordali4444/postgresql>

WHAT YOU ACHIEVED TODAY

You can now TRANSFORM data. Clean messy names, format dates for reports, calculate ages and durations, extract parts of strings, mask sensitive data. Data analysts do this 80% of the time — you now have the toolkit.

Tomorrow: CASE WHEN, COALESCE, NULLIF, CAST — conditional logic and type conversion. You will create calculated columns with IF-ELSE logic and handle NULL values like a pro.

Coming Tomorrow

What if you need different output based on conditions? CASE WHEN is like IF-ELSE for SQL. COALESCE replaces NULLs with defaults. CAST converts between types. Conditional Logic gives you decision-making power inside your queries.

Example preview: Instead of showing 0 or 6 for weekend, you will write:

```
CASE WHEN EXTRACT(DOW FROM date) IN (0, 6) THEN 'Weekend' ELSE 'Weekday' END
```

Turning numbers into meaningful labels!